KhidmatBot AI Service Orchestrator for Pakistan's Informal Economy

> Google Antigravity Hackathon — Challenge 2: AI Service Orchestrator for Informal Economy

---

What We Built

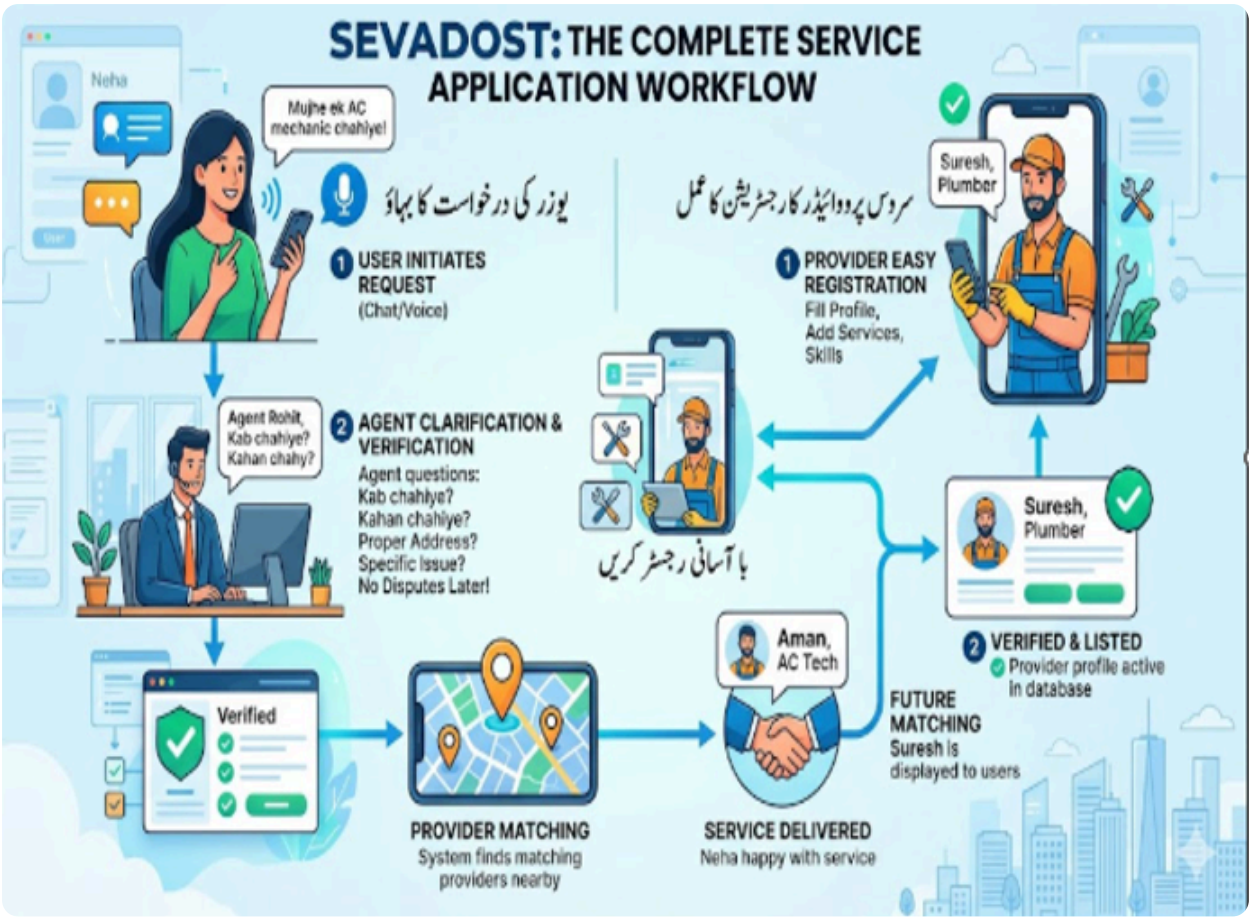
Most hackathon projects ship a demo. We built a real, production-ready platform. Any service provider plumber, electrician, AC technician, tutor anywhere in Pakistan can open the app, register with their NIC card, and start receiving AI-matched bookings immediately. The 8 providers in the seed dataset are demo data; every new real registration is treated identically by all 7 agents and all 13 ranking factors. **Three things we built beyond the challenge prompt:** | | | |---|---| | **Voice input in all three languages** | Users who cannot type can speak their request in Roman Urdu, Urdu script, or English — speech-to-text feeds directly into the NLU agent | | **Real NIC verification — no government API needed** | Provider photographs their NIC card → Google ML Kit reads the number via on-device OCR → matched against the typed NIC number. No mock database, no external service required | | **Full Pakistan coverage** | Not limited to any city or region — Google Maps handles real distance and travel time for any geocodable address anywhere in Pakistan |

Overview

Pakistan's informal service economy — plumbers, electricians, AC technicians, tutors, beauticians, drivers, mechanics — runs almost entirely on WhatsApp messages and phone calls. There is no structured discovery, no transparent pricing, no booking confirmation, and no dispute resolution. **KhidmatBot** is an agentic AI system that automates the full service lifecycle: from a natural-language request in Roman Urdu, Urdu, or English — all the way to booking confirmation, provider notification, service tracking, feedback, and dispute resolution. **Example:**

> User: "AC bilkul kaam nahi kar raha, kal subah G-13 mein technician chahiye, budget zyada nahi hai"

The system understands the request, ranks providers using 13 weighted factors, generates a transparent price quote, books the best match, notifies the provider, and follows up after the job.



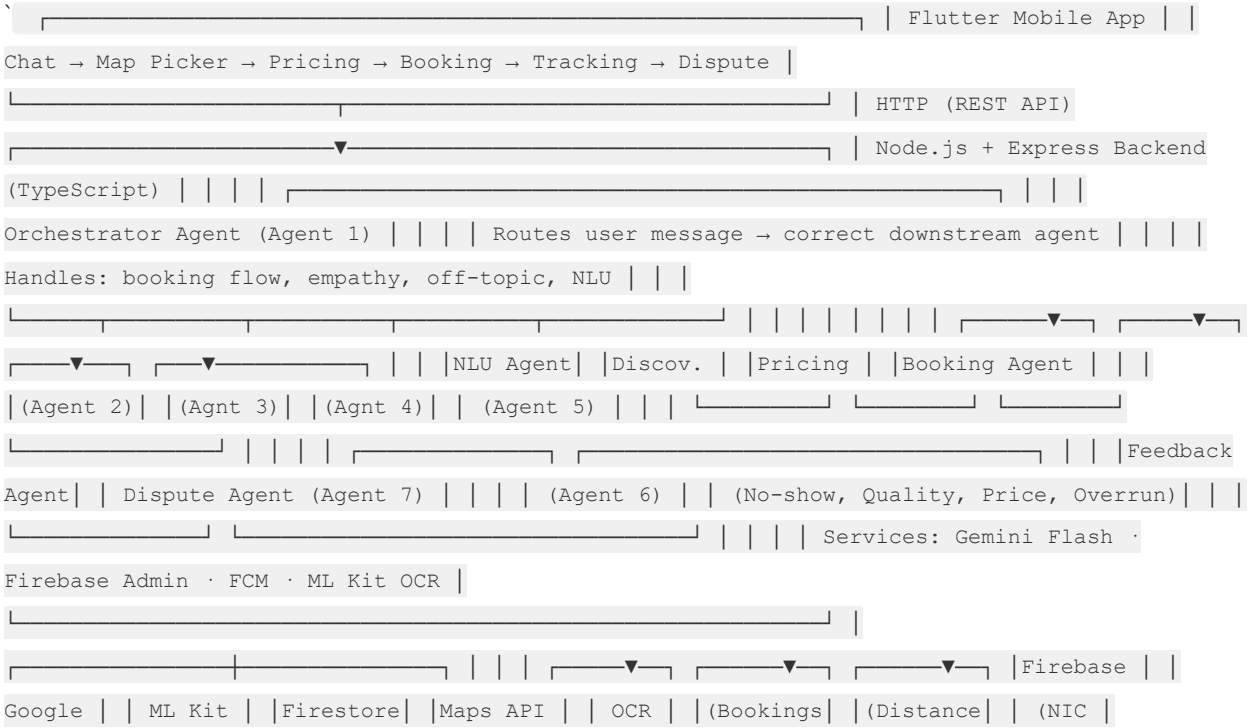
Antigravity Role and Workflow

Google Antigravity was used as the **primary development and orchestration platform** throughout this project. All agent logic, workflows, tool definitions, and agentic pipelines were designed, iterated, and tested inside Antigravity.

How Antigravity Orchestrates KhidmatBot

```
User Message | ▼ ┌──────────────────────────────────────┐ | Google Antigravity IDE | | | | Workplan → Task Plan → Agent Steps | | | | Step 1: NLU parsing | | → Tool: parse_multilingual_input | | → Output: intent JSON | | | | Step 2: Provider Discovery | | → Tool: search_providers | | → Tool: calculate_distance | | → Output: ranked provider list | | | | Step 3: Pricing | | → Tool: compute_price_components | | → Output: transparent breakdown | | | | Step 4: Booking | | → Tool: check_booking_conflict | | → Tool: create_booking | | → Tool: send_fcm_notification | | → Output: booking confirmation | | | | Fallback: No provider → waitlist | | Fallback: Conflict → next slot | | Fallback: API fail → area-match | └──────────────────────────────────────┘ Antigravity controlled: - **Intent understanding**: language detection → NLU structured extraction - **Matching and reasoning**: 13-factor ranking with visible rationale - **Scheduling decisions**: conflict check → slot suggestion → waitlist
```

- **Price logic**: component-by-component calculation with provider fairness - **Action execution**: booking creation → FCM notification → receipt - **Fallback behavior**: graceful degradation at every failure point ### Antigravity Traces All trace logs and screenshots are in antigravity_traces/ : | File | What It Shows | |------|---------------| | trace_01_language_confidence.png | Language parsing: Roman Urdu input → confidence score 87, entities extracted | | trace_02_ranking_rationale_part1.png | Provider ranking: 13-factor scoring for Ali Hassan vs Shahid Iqbal | | trace_02_ranking_rationale_part2.png | Ranking continued: WHY farther provider beats closer one | | 01_implementation_plan_open_questions.png | Antigravity workplan: initial architecture design session | | 02_architecture_summary_7_agents.png | Antigravity agent structure: 7 agents, handoffs, data flow | | 03_plan_update_in_progress.png | Scheduling intelligence decisions being reasoned | | 04_plan_fixes_applied.png | Price logic correction via Antigravity reasoning | | 05_nlu_agent_coding_started.png | Action execution: NLU agent implementation started | | 06_npm_init_success.png | Tool call success: backend initialized | Stress test traces in antigravity_artifact/ : - stress_test_run1.png — Agent pipeline run: waitlist + conflict scenarios - stress_test_run2.png — Agent pipeline run: cancel after accept + penalty - stress_test_run3.png — Agent pipeline run: misspelled input + price dispute --- ## Architecture



--- ## Agent Architecture — 7 Agents ### Agent 1 — Orchestrator Agent **File:** backend/src/agents/orchestrator.agent.ts The central router of the system. Every user message enters here first. - Detects language (English / Roman Urdu / Urdu script) and replies in the same language throughout the conversation - Routes to: Empathy Agent (frustrated users), Polite Decline Agent (off-topic), NLU flow (service requests) - Maintains conversation state: collects service type, location, datetime, urgency across multiple turns - Triggers the full pipeline (NLU → Discovery → Pricing → Booking) once all required fields are confirmed - Handles clarification loops: asks one follow-up question at a time **Sub-agents inside Orchestrator:** - **Empathy Agent** — handles angry/frustrated users, de-escalates before continuing - **Polite Decline Agent** — deflects off-topic questions (politics, weather,

cricket) --- ### Agent 2 — NLU Agent **File:** backend/src/agents/nlu.agent.ts Parses raw multilingual user input into structured intent. - Supports Roman Urdu, Urdu script, English, code-switched, slang, and misspellings - Uses Gemini Flash for semantic extraction - Outputs: service_type , location , datetime , urgency , budget_sensitive , job_complexity - Returns confidence score (0–100). If confidence < 70 → asks confirmation question in user's language - Resolves relative dates: "kal subah" → ISO datetime - Job complexity: basic / intermediate / complex **Sample output:** ` json { "service_type": "ac_repair", "location": "G-13", "datetime": "2026-05-20T10:00:00", "urgency": "high", "budget_sensitive": true, "job_complexity": "intermediate", "confidence": 87, "follow_up_needed": false } ` ![](<images for readmefile/agent2.png>) --- ### Agent 3 — Discovery & Ranking Agent **File:** backend/src/agents/discovery.agent.ts Finds and ranks available providers using 13 weighted factors. - Queries Firestore providers collection by service_type and area - Treats mock providers (is_mock: true) and real registered providers (is_mock: false) identically - Checks day-of-week availability before ranking - Applies 15-minute travel time buffer between provider bookings - Fallback if no provider: waitlist suggestion or next available date - Uses Google Maps / Directions API for real distance and travel time calculation **13-Factor Ranking Algorithm:** | # | Factor | Weight | |---|--------|--------| | 1 | Availability match (day + time slot) | 15% | | 2 | Distance / travel time | 12% | | 3 | Rating score | 12% | | 4 | Reliability / on-time score | 12% | | 5 | Review recency | 7% | | 6 | Review sentiment (positive/negative analysis) | 7% | | 7 | Skill specialization match | 10% | | 8 | Price vs budget sensitivity | 8% | | 9 | Capacity (slots available today) | 5% | | 10 | User preference (past booking history) | 5% | | 11 | Cancellation rate | 3% | | 12 | Risk score | 2% | | 13 | NADRA trust score (blue tick) | 2% | **Job complexity matching:** - basic → any provider - intermediate → 3+ years experience + tools available - complex → 5+ years + specialization + certifications required --- ### Agent 4 — Pricing Agent **File:** backend/src/agents/pricing.agent.ts Generates a transparent, fair dynamic price quote. **Price formula:** ` Base Rate (by complexity tier) + Urgency Fee (low: 0% | medium: 10% | high: 30% | emergency: 50%) + Distance Cost (cross-area: Rs.100 | same area: Rs.0) + Surge Fee (peak hours 12–15h, 18–21h: +10%) - Loyalty Discount (returning user: -5%) + Platform Fee (10% of subtotal, min Rs.100) = Total ` - Shows full breakdown per line item - Provider earning vs platform fee visible (fairness note) - Budget alternative option: simpler scope at lower price - Uses per-complexity rates (rate_basic , rate_intermediate , rate_complex) from provider data **Sample output:** ` json { "base_rate": 1200, "urgency_fee": 360, "distance_cost": 100, "surge_fee": 0, "loyalty_discount": 83, "platform_fee": 158, "total": 1735, "provider_earning": 1577, "breakdown_text": "Base: Rs.1200 + Urgency: Rs.360 + Distance: Rs.100 - Loyalty: Rs.83 + Platform: Rs.158 = Rs.1735", "provider_fairness": "Provider receives 91% of total" } ` ![](<images for readmefile/agent4.png>) --- ### Agent 5 — Booking Agent **File:** backend/src/agents/booking.agent.ts Handles end-to-end booking simulation. - Checks for double-booking conflicts before confirming - Creates booking in Firestore with pending status (provider must accept) - Sends FCM push notification to provider's device - Simulates WhatsApp message (Pakistan's primary notification channel) - Generates receipt: booking_id , provider details, service, datetime, price - Conflict resolution: if slot taken → suggest next free slot or add to waitlist - **Accept-then-reject penalty flow**: if provider accepts and then cancels: - Client is immediately notified - Auto-reschedule: next best available provider found - Provider profile updated: cancellation_rate +1 , reliability_score -10 , risk_score

escalates - "Cancel ke baad cancel" warning banner visible on provider profile ![](<images for readmefile/notification.png>) --- ### Agent 6 — Feedback Agent **File:** backend/src/agents/feedback.agent.ts Manages the service quality loop after job completion. - En-route update simulation: "Ali Hassan nikal gaya, 12 min mein pohonchega" - Service completion checklist: arrived on time, work done, area cleaned, customer satisfied - Collects 1–5 star rating + free-text comment - Updates provider's rating in Firestore (weighted average) - Logs ranking impact: good feedback → reliability_score boost; bad → penalty --- ### Agent 7 — Dispute & Resolution System **Files:** backend/src/agents/dispute-report.agent.ts · backend/src/agents/dispute.agent.ts The dispute system uses a **two-agent + human-in-the-loop** design. The AI does not make final decisions unilaterally — it prepares a neutral analysis and the human team makes the final call. **Full dispute flow:** ` Customer submits complaint (+ evidence photos) | ▼ POST /dispute/analyze | ▼ ┌──────────────────────────────────┐ | Dispute Report Agent | | (dispute-report.agent.ts) | | | | Reads: user_complaint | | provider_response | | evidence_photos[] | | original_price | | | | Produces neutral structured report: | | - Summary of dispute | | - Client's perspective (objective) | | - Provider's perspective (objective) | | - Evidence photo analysis | | - Key discrepancies (fact clashes) | | - Severity: low / medium / high | | - Recommended action for team | └──────────────┬───────────────────┘ | ▼ Firestore: disputes/{id} status = "pending_team_decision" ai_report = { full report } | ▼ Email sent to team (nodemailer / Gmail) Subject: "[Dispute] {id} — AI Report Tayyar" Body: All report sections in formatted HTML | ▼ Human team reviews → makes final decision ` **What the Dispute Report Agent does NOT do:** - Does not apply strikes or penalties directly - Does not issue refunds unilaterally - Does not take sides — presents both parties' claims objectively **Dispute types supported:** | Type | What the AI reports | Severity typical | |---|---|---| | **No-show** | Provider did not arrive — full refund recommended | High | | **Quality complaint** | Work done but substandard — partial refund recommended (20%, or lower if provider defense is valid) | Medium | | **Price disagreement** | Provider charged more than quoted — overcharge amount noted | Medium | | **Overrun** | Job took longer — extra charge requires customer approval first | Low–Medium | | **Cancellation** | Customer cancelled — full refund if 2h+ before job, 10% fee if < 2h | Low | **Blacklist logic** (in dispute.agent.ts): When the team applies a strike via apply_provider_strike , the tool checks the total count. At 3 strikes → provider is permanently removed from the platform. - apply_provider_strike — increments strike count in Firestore, returns strikes_after ; if ≥ 3, status = blacklisted - apply_provider_penalty — reduces reliability_score and ranking weight for quality complaints - All dispute records saved in Firestore disputes collection with timestamps - Evidence photos accepted as URLs (uploaded via Firebase Storage from the dispute screen) --- ## Flutter Mobile App — 16 Screens | Screen | File | Description | |--------|------|-------------| | Home Screen | home_screen.dart | Landing page with service categories | | Chat Screen | chat_screen.dart | Main AI chat with reasoning panel | | Map Picker | map_picker_screen.dart | Google Maps location pin drop | | Pricing Screen | pricing_screen.dart | Full price breakdown + budget option | | Booking Confirmation | booking_confirmation_screen.dart | Receipt with countdown timer | | Booking Waiting | booking_waiting_screen.dart | Provider response pending screen | | Booking Status | booking_status_screen.dart | Live status after provider accepts | | Service Tracking | service_tracking_screen.dart | En-route + completion checklist | | Feedback Screen | feedback_screen.dart | Star rating + comment submission | | Dispute Screen |

dispute_screen.dart | Dispute type selection + submission | | Provider Profile | provider_profile_screen.dart | Full profile: photo, reviews, on-time score | | Provider Registration | provider_registration_screen.dart | New provider onboarding + NADRA check | | Provider Notification | provider_notification_screen.dart | Accept/decline booking request | | NIC Scanner | nic_scanner_screen.dart | ML Kit OCR for NIC number extraction | | Agent Trace | agent_trace_screen.dart | Antigravity reasoning trace viewer | | Baseline Comparison | baseline_comparison_screen.dart | Old method vs KhidmatBot comparison | ### Reasoning Panel (Chat Screen) Before any provider result appears, an animated reasoning card shows: ` Soch raha hoon... ✓ Samajh gaya: AC repair, G-13, kal subah ✓ 3 providers mile aapke area mein ✓ 13 factors check kar raha hoon... → Ali Hassan: Score 87/100 Available ✓ | 1.2km ✓ | Rating 4.8 ✓ → Tariq Mehmood: Score 71/100 ✓ Decision: Ali Hassan best match ` ![](<images for readmefile/reasoning panel.png>) --- ## Provider Dataset Schema `json { "id": "p1", "name": "Ali Hassan", "photo_url": "assets/providers/ali_hassan.jpg", "service_types": ["ac_repair"], "nic": "4210112345671", "blue_tick": true, "rating": 4.8, "total_reviews": 121, "review_sentiment": "positive", "experience_years": 4, "certifications": ["HVAC Tech"], "tools_available": ["Gauge Manifold", "Vacuum Pump"], "area": "G-11", "coordinates": { "lat": 33.673, "lng": 73.013 }, "hourly_rate": 800, "rate_basic": 700, "rate_intermediate": 1200, "rate_complex": 2000, "availability": { "monday": ["09:00", "14:00"], "tuesday": ["10:00"], "wednesday": ["09:00", "15:00"], "thursday": [], "friday": ["14:00"], "saturday": ["10:00", "16:00"], "sunday": [] }, "on_time_score": 92, "cancellation_rate": 7, "capacity_today": 3, "risk_score": "low", "reliability_score": 95, "strikes": 0, "user_preference_score": 0, "is_mock": true, "registered_at": "2024-01-15T00:00:00Z" } ` ### 8 Mock Providers | Name | Service | Area | NADRA Verified | |------|---------|------|----------------| | Ali Hassan | AC Repair | G-11 | ✅ Blue Tick | | Tariq Mehmood | Electrician | G-13 | ✅ Blue Tick | | Bilal Ahmed | Plumber | F-10 | ✅ Blue Tick | | Usman Ali | Carpenter | F-8 | ✅ Blue Tick | | Sana Malik | Tutor | I-8 | ✅ Blue Tick | | Aslam Khan | Plumber | G-13 | ✅ Blue Tick | | Kamran Shah | Electrician | F-10 | ❌ Unverified | | Shahid Iqbal | AC Tech | G-11 | ❌ Unverified | --- ## NADRA NIC Verification Providers optionally verify their identity during registration using their National Identity Card. No government API or external service is required — verification runs entirely on-device using Google ML Kit. **Verification flow:** 1. Provider uses the in-app NIC Scanner to photograph their NIC card 2. Provider types their NIC number manually into the registration form 3. Google ML Kit (on-device OCR) reads the NIC number from the card photo 4. System compares the OCR-extracted number against the typed number 5. Match → blue_tick: true , trust score boost applied in Discovery ranking 6. No match or skipped → blue_tick: false , no penalty (verification is optional) The blue tick appears on the provider's profile card and contributes a 2% weight in the 13-factor ranking algorithm. When multiple providers score closely, verified providers are preferred. The 8 seed providers in the dataset have pre-assigned blue_tick values for demo purposes; every newly registered real provider goes through this live OCR flow. --- ## APIs and Tools Used | API / Service | Purpose | |---|---| | **Google Antigravity** | Primary orchestration platform — designed and drove all agent workflows, task plans, tool calls, and agentic pipelines | | **OpenAI Agents SDK** (@openai/agents) | Agent runtime framework (used within Antigravity-designed workflows) | | **GPT-4o-mini** | Agent reasoning engine (Discovery, Pricing, Booking, Dispute) | | **Gemini Flash** (@google/generative-ai) | NLU parsing, JSON extraction | | **Firebase Firestore** | Provider data, bookings, disputes, user sessions | | **Firebase Storage** | Provider photo

uploads | | **Firebase Cloud Messaging (FCM)** | Push notifications to providers | | **Google Maps Flutter** | Location pin drop, nearby provider markers | | **Google Speech-to-Text** | Voice input in chat screen | | **Google ML Kit Text Recognition** | NIC number OCR scanning + on-device verification | | **Express.js** | REST API server | | **Zod** | Runtime schema validation | --- ## Multilingual Support The system handles 3 language modes with automatic detection: | Language | Example Input | System Response | |---|---|---| | **English** | "I need a plumber tomorrow" | Full English reply | | **Roman Urdu** | "kal subah G-13 mein plumber chahiye" | Full Roman Urdu reply | | **Urdu Script** | "مجھے کل پلمبر چاہیے" | Full Urdu script reply | | **Mixed** | "mujhe AC fix karwana hai ASAP" | Matches dominant language | - Slang normalized: "jldi" → "jaldi", "krdo" → "karo" - Misspellings handled via Gemini's semantic understanding - Confidence score: if < 70 → asks confirmation before proceeding - Language detection is enforced at the Orchestrator level — never mixes languages in a single response --- ## Scheduling Intelligence - **Double booking prevention**: `check_booking_conflict` tool queries Firestore before confirming - **15-minute travel buffer**: between consecutive bookings for the same provider - **Alternate slot suggestion**: `find_next_free_slot` tool finds next available time if conflict exists - **Waitlist**: if no provider available → user added to waitlist, notified when slot opens - **Auto-reschedule**: if provider cancels after accepting → system immediately runs Discovery + Ranking again and assigns next best provider --- ## Robustness and Fallbacks | Scenario | Handling | |---|---| | No provider available | Waitlist offered + next available date suggested | | Low confidence input (< 70) | Confirmation question asked in user's language | | API failure (Maps/Gemini) | Graceful error, fallback to area-match instead of GPS distance | | Payment confirmation failure | Retry logic (3 attempts), booking held, user notified | | Provider no response (5 min) | Auto-decline, next provider notified | | User preference conflicts | Agent asks clarifying question before proceeding | | Off-topic message | Polite decline with service examples | | Angry/frustrated user | Empathy agent de-escalates before service flow | --- ## Stress Test Results | Test | Scenario | Result | |---|---|---| | 1 | No provider available in time window | ✅ Waitlist triggered, next slot suggested | | 2 | Provider cancels after confirmation | ✅ Auto-reschedule, penalty applied, client notified | | 3 | Misspelled / mixed-language input | ✅ Confidence 70, confirmation asked | | 4 | Two users book same provider same slot | ✅ Second user gets conflict, next slot offered | | 5 | Customer disputes price after service | ✅ Exact overcharge refunded | | 6 | High rating but recent bad reviews + high cancellation rate | ✅ review_sentiment 40/100, ranking drops significantly | --- ## Baseline Comparison | Capability | Traditional (WhatsApp/Phone) | KhidmatBot | |---|---|---| | Service discovery | Manual referrals, unknown quality | AI-ranked, 13-factor matching | | Pricing | Negotiated verbally, unpredictable | Dynamic, transparent, line-item breakdown | | Language support | Any (human handles it) | Urdu, Roman Urdu, English, mixed | | Booking confirmation | Verbal only, often forgotten | Firestore record + FCM notification + receipt | | Provider verification | Reputation only (word of mouth) | NADRA NIC check + Blue Tick badge | | Double booking prevention | None | Automated conflict detection | | Dispute resolution | No process | 5-type automated resolution with refund logic | | Provider ranking | Whoever picks up first | 13-factor weighted algorithm | | Follow-up | Forgotten | En-route update + completion checklist + rating | | Penalty for bad behavior | None | Strike system → automatic removal at 3 strikes | --- ## Cost and Latency Estimate ### Per-Request Cost (1 booking flow) | Step | Model/API | Est. Tokens | Est. Cost | |---|---|---|---| | NLU parsing | Gemini Flash

| ~800 tokens | ~$0.00008 | | Discovery + Ranking | GPT-4o-mini | ~1,200 tokens | ~$0.00024 | | Pricing generation | GPT-4o-mini | ~800 tokens | ~$0.00016 | | Booking confirmation | GPT-4o-mini | ~600 tokens | ~$0.00012 | | **Total per booking** | | ~3,400 tokens | **~$0.0006** | At 10,000 bookings/day → ~$6/day in AI costs. ### Latency (end-to-end booking) | Stage | Time | |---|---| | NLU parsing | ~1.5s | | Provider discovery | ~1.0s | | Pricing calculation | ~1.5s | | Booking creation | ~0.8s | | **Total** | **~5s** | ### Scaling Plan - **10x (100K bookings/day)**: Horizontal scaling of Express server, Firestore auto-scales, AI costs ~$60/day - **100x (1M bookings/day)**: Move to Cloud Run + load balancer, implement Redis caching for provider data, batch FCM notifications, AI costs ~$600/day - Provider data cached in memory with 5-minute TTL to reduce Firestore reads - Agent responses for identical requests can be cached (same area + service type) --- ## Privacy Note - NIC numbers are used only for NADRA verification during provider registration and are never returned in API responses to customers - Provider NIC data is stored encrypted in Firestore - Customer location data (GPS coordinates) is used only for distance calculation and is not stored permanently - NIC card photos captured during registration are processed on-device by Google ML Kit and are not uploaded to or stored on the server - Photos uploaded via Firebase Storage are accessible only via signed URLs - No customer financial data is stored — payments are simulated only --- ## Assumptions and Limitations **Assumptions:** - The 8 seed providers in the dataset are from Islamabad sectors (G-11, G-13, F-10, F-8, I-8); real providers from any city or region in Pakistan can register and are fully supported - Payment processing is simulated — no real payment gateway integrated - WhatsApp notifications are simulated — no real WhatsApp Business API key used - Voice input requires microphone permission on device **Limitations:** - Currently supports 8 service types: AC repair, electrician, plumber, carpenter, tutor, beautician, driver, mechanic - Real-time provider location tracking (en-route GPS) is simulated with estimated time - ML Kit NIC OCR works best in good lighting conditions - No payment processing — booking cost is displayed but not charged - Background notifications on iOS require additional APNs certificate configuration --- ## Local Setup ### Prerequisites - Node.js 18+ - Flutter 3.10+ - Android Studio / Android SDK - Firebase project with Firestore, Storage, FCM enabled - OpenAI API key - Gemini API key - Google Maps API key (Maps JavaScript, Directions, Places, Geocoding) ### Backend Setup ` bash # Clone the repository git clone https://github.com/sadafcode/anti_hackathon_project cd anti_hackathon_project # Install dependencies npm install # Configure environment variables cp .env.example .env # Edit .env and add your API keys: # OPENAI_API_KEY= # GEMINI_API_KEY= # MAPS_API_KEY= # Add Firebase service account # Download serviceAccountKey.json from Firebase Console # Place in project root # Start backend server npm run dev # Server runs at http://localhost:3000 ` ### Flutter App Setup ` bash cd app # Install Flutter dependencies flutter pub get # Configure Firebase # Add google-services.json to app/android/app/ # Add GoogleService-Info.plist to app/ios/Runner/ # Add Google Maps API key # In app/android/app/src/main/AndroidManifest.xml: # <meta-data android:name="com.google.android.geo.API_KEY" # android:value="YOUR_MAPS_API_KEY"/> # Run the app flutter run ` ### Environment Variables ` OPENAI_API_KEY= # For agent reasoning (GPT-4o-mini) GEMINI_API_KEY= # For NLU parsing (Gemini Flash) MAPS_API_KEY= # Google Maps / Directions / Places FIREBASE_PROJECT_ID= # Firebase project ID ` --- ## Repository Structure ` anti_hackathon_project/ ├── backend/ | ├── data/ | | └── providers.json # 8 mock providers dataset | └── src/ | ├── agents/ | | ├── orchestrator.agent.ts # Agent 1 — Router + Conversation | | ├── nlu.agent.ts # Agent 2 — Language Understanding | | ├──

```
discovery.agent.ts # Agent 3 — Provider Discovery + Ranking | | ├── pricing.agent.ts # Agent 4 — Dynamic Pricing | | ├── booking.agent.ts # Agent 5 — Booking + Scheduling | | ├── feedback.agent.ts # Agent 6 — Quality + Feedback | | ├── dispute.agent.ts # Agent 7 — Dispute Resolution | | └── schemas.ts # Zod schemas for all agent I/O | ├── services/ | | ├── gemini.service.ts # Gemini Flash wrapper | | ├── fcm.service.ts # Firebase Cloud Messaging | | ├── nadra.service.ts # Mock NADRA verification | | └── session.service.ts # Conversation session store | ├── tools/ | | ├── provider.tools.ts # search_providers, apply_strike | | └── booking.tools.ts # check_conflict, create_booking | ├── prompts/ | | └── nlu.prompt.ts # NLU few-shot prompt builder | ├── routes/ | | └── api.routes.ts # All REST endpoints | └── app.ts # Express app entry point ├── app/ | └── lib/ | ├── screens/ # 16 Flutter screens | ├── services/ | | └── api_service.dart # All backend API calls | ├── models/ # Dart data models | ├── widgets/ # Reusable UI components | └── main.dart ├── .env.example └── README.md
```

Key Differentiators

- **NADRA NIC verification + Blue Tick** — optional for real providers, verifiable trust signal
- **13-factor weighted ranking** with visible reasoning — user sees exactly why a provider was selected
- **Roman Urdu + slang + misspellings + voice input** with confidence scoring
- **Real provider registration** — any service professional can join; treated identically to mock providers in all operations
- **Live reasoning panel in chat** — decision process animated and visible before result appears
- **Accept-then-reject penalty** — provider profile suffers permanently if they cancel after accepting
- **Dynamic transparent pricing** — per line item, provider fairness % shown
- **Full dispute handling** — 5 types with automatic refund logic and strike tracking
- **WhatsApp notification simulation** — Pakistan's primary communication channel
- **3-strike system** — automatic platform removal for repeated bad behavior
- **Empathy agent** — handles frustrated users before entering service flow
- **NIC OCR scanner** — camera-based NIC number extraction for faster registration

---

Built for Google Antigravity Hackathon — Challenge 2 | Team: Sadaf